Note 2: Writing a Dockerfile for a Maven Application

Before Using Any Plugin — Understanding the Dockerfile

Regardless of which plugin you use, you need to understand how a Java Maven application is containerized. The Dockerfile is the blueprint for the Docker image.

Basic Dockerfile for a Maven Spring Boot Application

```
# Stage 1: Build the application

FROM maven:3.9.4-eclipse-temurin-17 AS builder


WORKDIR /app


# Copy the POM file first (for dependency caching)

COPY pom.xml .


# Download all dependencies (this layer is cached if pom.xml doesn't change)

RUN mvn dependency:go-offline -B


# Copy the source code

COPY src ./src


# Build the application, skipping tests

RUN mvn package -DskipTests


# Stage 2: Create the runtime image

FROM eclipse-temurin:17-jre-alpine


WORKDIR /app


# Copy only the JAR from the build stage
```

```
COPY --from=builder /app/target/*.jar app.jar


# Expose the application port

EXPOSE 8080


# Run the application

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Why Multi-Stage Builds?

A multi-stage Dockerfile separates the build environment from the runtime environment:

- Stage 1 (builder) uses a full Maven + JDK image which is large (hundreds of MB) but has everything needed to compile and package the application
- Stage 2 (runtime) uses a minimal JRE-only image (Alpine-based, around 80MB) that only has what is needed to run the JAR

Without multi-stage builds, your final Docker image would contain Maven, the JDK, all source code, and all downloaded dependencies — making it unnecessarily large and exposing build tooling in production.

Layer Caching Optimisation

Notice the order of COPY commands in the Dockerfile:

```
COPY pom.xml .

RUN mvn dependency:go-offline -B

COPY src ./src

RUN mvn package -DskipTests
```

This order is deliberate. Docker caches each layer. By copying pom.xml first and downloading dependencies before copying the source code, the dependency download layer is only re-executed when pom.xml changes. If only the source code changes, Docker reuses the cached dependency layer, making rebuilds significantly faster.

If you copy everything at once (COPY . .), every source code change invalidates the dependency cache and Maven re-downloads all dependencies every time.

Single-Stage Dockerfile (Simpler, Larger Image)

For development or learning purposes, a single-stage build is simpler:

FROM eclipse-temurin:17-jdk-alpine

WORKDIR /app

COPY target/my-app-1.0.0.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]

This assumes you have already run mvn package locally and the JAR exists in target/. The Dockerfile simply copies it in and sets up the run command. This is the approach used with the dockerfile-maven-plugin — Maven handles the build, and the Dockerfile handles the image creation.

_____